

# **C Programming Mini Project**

## **Digital Assignment II**

### **Employee Payroll System**

**Student Name: Nikhil Thomas**

**Register Number: 25BCE5061**

**Department: Computer Science and Engineering**

**Course Name: BACSE104**

**Faculty Name: DINAKARAN M**

**Submission Date: 10/04/2026**

**GitHub Repository Link:**

**<https://github.com/nikhilthomas2025/DA-II BACSE104>**

## **Problem Statement:**

Many small and medium-sized businesses use manual entry or simple spreadsheets to figure out how much to pay their employees each month. This method is very likely to lead to mistakes by people, especially when dealing with changing overtime hours and tax brackets that get higher as you earn more money. Errors in these calculations can cause problems with payroll, make employees unhappy, and put you at risk of not following tax laws.

### **1. Features Implemented:**

- Add Employee Details
- Alter Overtime hours
- Generate Payroll Slip
- View Highest paid Employee
- Input validation

### **2. System Design:**

#### **2.1. Class Definition:**

```
class Employee {  
private:  
    int empID;  
    string name;  
    double basicPay;  
    double otHours;  
    const double OT_RATE = 25.0;  
  
public:  
    Employee(int id, string n, double bp, double ot)  
    double calculateGross()  
    double calculateTax()  
    double calculateNetPay()  
    int getID()  
    string getName()  
    void setOT(double hours)  
    void displaySlip()
```

|                 |                                                         |
|-----------------|---------------------------------------------------------|
| <b>empID</b>    | <b>Unique ID to track every employee</b>                |
| <b>name</b>     | <b>Contains the First and Last name of the employee</b> |
| <b>basicPay</b> | <b>Contains the base salary of the employee</b>         |
| <b>otHours</b>  | <b>Contains the hours of overtime of the employee</b>   |

```
class PayrollSystem {
```

```
    private:
```

```
        vector<Employee> employees;
```

```
    public:
```

```
        void addEmployee()
```

```
        void updateOT()
```

```
        void generatePayslip()
```

```
        void showReports()
```

```
};
```

|                  |                                    |
|------------------|------------------------------------|
| <b>employees</b> | <b>Stores all employee records</b> |
|------------------|------------------------------------|

## 2.2. Function List:

| Function Name            | Purpose                           | Inputs                                     | Outputs             |
|--------------------------|-----------------------------------|--------------------------------------------|---------------------|
| calculateGross()         | Calculates Gross pay amount       | Basic pay, OverTime hours                  | Gross pay           |
| calculateTax()           | Calculates Tax amount             | Gross Pay                                  | Tax amount          |
| calculateNetPay()<br>( ) | Calculates final salary after tax | Gross pay, Tax                             | Final Salary amount |
| getID()                  | Returns ID                        | ID                                         | ID                  |
| getName()                | Returns Name                      | Name                                       | Name                |
| setOT()                  | Returns Overtime hours            | Overtime hours                             | Overtime hours      |
| displaySlip()            | Displays payslip                  | ID,Name,OT hours, Gross Pay<br>Tax,Net pay | Printslip           |

| Function Name     | Purpose                 | Inputs            | Outputs                      |
|-------------------|-------------------------|-------------------|------------------------------|
| addEmployee()     | Adds Employee record    | ID,Name,OT hours, | Success Message              |
| updateOT()        | Updates Over time hours | ID,new OT hours   | Success Message              |
| generatePayslip() | Generates Payslip       | ID                | Payslip                      |
| showReports()     | Shows Highest Salary    | none              | Employee with Highest Salary |

### **3. Program Flow Explanation:**

- Main menu: Displays all 5 possible Choices to the user. The user enters the corresponding number to select a given choice.
- Add Employee: Takes ID, name, basic pay, OverTime hours and creates the record
- Update OT: Takes the OverTime hours and updates entry of the employee
- Generate Payslip: Takes the ID and Displays the Pay Slip.
- Show Report: Displays the Details of highest Salary employee.
- Exit: Exits the menu and ends the program.

### **4. Core Logic Explanation:**

- Core logic revolves around ID to search and edit each Employee.
- Checks for existence of ID every time.

## 5. Screenshots:

```
===== PAYROLL MENU =====
1. Add Employee
2. Update OT
3. Generate Payslip
4. Show Report
0. Exit
Enter choice: 1
Enter ID: 101
Enter Name: Nikhil Thomas
Enter Basic Pay: 1000.05
Enter OT Hours: 15
Employee added successfully.
```

```
===== PAYROLL MENU =====
1. Add Employee
2. Update OT
3. Generate Payslip
4. Show Report
0. Exit
Enter choice: 1
Enter ID: 401
Enter Name: First son
Enter Basic Pay: 1500.50
Enter OT Hours: 5
Employee added successfully.
```

```
===== PAYROLL MENU =====
1. Add Employee
2. Update OT
3. Generate Payslip
4. Show Report
0. Exit
Enter choice: 2
Enter EmpID to update OT: 101
Enter new OT Hours: 27
OT Updated.
```

```
===== PAYROLL MENU =====
1. Add Employee
2. Update OT
3. Generate Payslip
4. Show Report
0. Exit
Enter choice: 3
Enter EmpID: 101

----- PAYSALIP -----
ID: 101 | Name: Nikhil Thomas
Basic Pay: $1000.05
OT Hours: 27.00 (Rate: $25.00)
Gross Pay: $1675.05
Tax Ded.: $0.00
Net Pay: $1675.05
-----

===== PAYROLL MENU =====
1. Add Employee
2. Update OT
3. Generate Payslip
4. Show Report
0. Exit
Enter choice: 4

===== MONTHLY REPORT =====
Total Monthly Payout: $3300.55
Highest Paid: Nikhil Thomas ($1675.05)

===== PAYROLL MENU =====
1. Add Employee
2. Update OT
3. Generate Payslip
4. Show Report
0. Exit
Enter choice: 0
Exiting...
```

## 6. Deployment Explanation:

Entire project is on github with the source code and exe file which can be downloaded and run natively.

## 7. Challenges Faced:

Unable to get the C code to be compatible with WebAssembly and modern browser website norms. Norms of C conflict with norms of websites which cause errors, crashes, and inefficiencies.

## 8. AI Usage Declaration:

### Gemini:

Basic structure and syntaxing

## 9. Complete Source Code:

```
#include <iostream>
#include <vector>
#include <string>
#include <iomanip>
#include <algorithm>

using namespace std;

class Employee {
private:
    int empID;
    string name;
    double basicPay;
    double otHours;
    const double OT_RATE = 25.0;

public:
    Employee(int id, string n, double bp, double ot)
        : empID(id), name(n), basicPay(bp), otHours(ot) {}

    // Calculation Methods
    double calculateGross() const {
        return basicPay + (otHours * OT_RATE);
    }

    double calculateTax() const {
        double gross = calculateGross();
        if (gross <= 2000) return 0;
        if (gross <= 5000) return (gross - 2000) * 0.10;
```

```

        return (3000 * 0.10) + (gross - 5000) * 0.20;
    }

    double calculateNetPay() const {
        return calculateGross() - calculateTax();
    }

    // Getters & Setters
    int getID() const { return empID; }
    string getName() const { return name; }
    void setOT(double hours) { otHours = hours; }

    void displaySlip() const {
        cout << "\n----- PAYSALIP -----" << endl;
        cout << "ID: " << empID << " | Name: " << name << endl;
        cout << "Basic Pay: $" << fixed << setprecision(2) << basicPay << endl;
        cout << "OT Hours:  " << otHours << " (Rate: $" << OT_RATE << ")" << endl;
        cout << "Gross Pay: $" << calculateGross() << endl;
        cout << "Tax Ded.:  $" << calculateTax() << endl;
        cout << "Net Pay:   $" << calculateNetPay() << endl;
        cout << "-----" << endl;
    }
};

class PayrollSystem {
private:
    vector<Employee> employees;
public:
    void addEmployee() {
        int id;
        string name;
        double bp, ot;

        cout << "Enter ID: "; cin >> id;

        for (const auto& e : employees) {
            if (e.getID() == id) {
                cout << "Error: ID already exists!\n";
                return;
            }
        }

        cout << "Enter Name: "; cin.ignore(); getline(cin, name);
    }
};

```



```

    cout << "Enter Basic Pay: "; cin >> bp;
    cout << "Enter OT Hours: "; cin >> ot;

    if (bp < 0 || ot < 0) {
        cout << "Error: Pay/Hours cannot be negative.\n";
    } else {
        employees.emplace_back(id, name, bp, ot);
        cout << "Employee added successfully.\n";
    }
}

void updateOT() {
    int id;
    double hours;
    cout << "Enter EmpID to update OT: "; cin >> id;
    for (auto& e : employees) {
        if (e.getID() == id) {
            cout << "Enter new OT Hours: "; cin >> hours;
            if (hours >= 0) {
                e.setOT(hours);
                cout << "OT Updated.\n";
            } else cout << "Invalid hours.\n";
            return;
        }
    }
    cout << "Employee not found.\n";
}

void generatePayslip() {
    int id;
    cout << "Enter EmpID: "; cin >> id;
    for (const auto& e : employees) {
        if (e.getID() == id) {
            e.displaySlip();
            return;
        }
    }
    cout << "Employee not found.\n";
}

void showReports() {
    if (employees.empty()) return;

```

```

double totalPayout = 0;
Employee* highest = &employees[0];

for (auto& e : employees) {
    double net = e.calculateNetPay();
    totalPayout += net;
    if (net > highest->calculateNetPay()) {
        highest = &e;
    }
}

cout << "\n===== MONTHLY REPORT =====" << endl;
cout << "Total Monthly Payout: $" << totalPayout << endl;
cout << "Highest Paid: " << highest->getName() << " ($" << highest-
>calculateNetPay() << ")" << endl;
}
};

int main() {
    PayrollSystem ps;
    int choice;

    do {
        cout << "\n===== PAYROLL MENU =====\n";
        cout << "1. Add Employee\n";
        cout << "2. Update OT\n";
        cout << "3. Generate Payslip\n";
        cout << "4. Show Report\n";
        cout << "0. Exit\n";
        cout << "Enter choice: ";
        if (!(cin >> choice)) {
            cout << "Invalid input! Please enter a number." << endl;
            cin.clear(); // Clear error flags
            cin.ignore(1000, '\n'); // Discard buffer
            continue;
        }

        switch (choice) {
            case 1: ps.addEmployee(); break;
            case 2: ps.updateOT(); break;
            case 3: ps.generatePayslip(); break;
            case 4: ps.showReports(); break;
            case 0: cout << "Exiting...\n"; break;
        }
    }
}

```

```
        default: cout << "Invalid choice.\n";
    }
} while (choice != 0);

return 0;
}
```

## **10. Conclusion:**

This project shows how Object-Oriented Programming (OOP) can be used to solve real-world administrative payroll problems. I made the switch from a manual payroll process to an automated C++ system. This made my data more secure through encapsulation and made my system more scalable by using the Standard Template Library (STL).

LLM AIs has helped tweak and catch mistakes saving on human time and energy.